

NLP Project

Categorize text in the Arabic language.

NAMES	ID
Antonios Gerges Nageh	20221903971
Abdelrhman Mohamed Abdelhady	2022513643
Abdelrhman Tarik Zaki	20221442265

Introduction

In the era of vast digital content, the efficient categorization of textual information is imperative for numerous applications. This project delves into the domain of Natural Language Processing (NLP) to address the challenges of categorizing Arabic text into distinct and meaningful categories. The primary objective is to develop models capable of accurately assigning texts to predefined categories, facilitating streamlined information retrieval and analysis.

Project Scope and Categories

The text categorization undertaken encompasses a diverse range of subjects, reflecting the richness and variety of Arabic discourse. The identified categories include:

- Tech
- Sports
- Religion
- Politics
- Medical
- Finance
- Culture

These categories were carefully chosen to represent broad domains, allowing the models to capture the nuances and intricacies of content across various topics.

Models Employed

The project leverages three distinct models to undertake the text categorization task:

1. Vanilla RNN Model:

- A baseline recurrent neural network (RNN) model was employed, providing a foundation for comparison with more sophisticated architectures.
- The model's performance is assessed based on its ability to capture sequential dependencies in the data.

2. LSTM Model:

- Long Short-Term Memory (LSTM) networks, known for their enhanced ability to capture long-range dependencies, were utilized.
- The training and evaluation of the LSTM model aimed at exploring improvements in text categorization accuracy.

3. Multinomial Naive Bayes Model:

- A probabilistic and straightforward Multinomial Naive Bayes model was incorporated as an additional benchmark.
- This model offers a different paradigm, relying on probabilistic assumptions to classify texts into predefined categories.

Project Objectives

The overarching goal of this project is to assess the effectiveness of the chosen models in accurately categorizing Arabic text across diverse topics. Through the implementation of Vanilla RNN, LSTM, and Multinomial Naive Bayes models, we aim to not only achieve high categorization accuracy but also gain insights into the strengths and limitations of each approach.

As we navigate through the subsequent sections, a detailed account of our data preprocessing steps, model architectures, training processes, and performance evaluations will be presented, providing a comprehensive understanding of the methodologies employed and the outcomes achieved.

Data Collection and Organization

1. Data Paths:

- The project utilized a dataset consisting of text files, categorized into different subjects such as Culture, Tech, Sports, Religion, Politics, Medical, and Finance. Each category had its specific path.

2. Loading and Organizing Data:

- A function was created to systematically load and preprocess data from the specified paths.
- The function iterated through each category, reading text files and assigning labels to represent the corresponding categories.

Text Extraction and Storage

3. Text Extraction:

- For each file in a category, the text was extracted by reading the contents of the file using an appropriate encoding.

4. Label Assignment:

- Labels were assigned to each text based on the category it belonged to. Numerical representations were used for each category, facilitating model training.

Data Splitting

5. Train-Validation-Test Split:

- The dataset was divided into training, validation, and test sets to assess model performance effectively.
- Parameters controlled the ratio of data allocated for testing and the portion of the training data used for validation.

6. Randomization:

- To ensure the models' robustness, the data splitting process involved randomization for reproducibility.

Text Preprocessing

In the text processing phase, raw text data was transformed into a structured format suitable for model input through a series of steps:

Tokenization and Padding

1. Tokenization:

- Tokenization is the process of converting text into a sequence of tokens or words. In this project, a maximum of 10,000 words was considered for tokenization.
- The **Tokenizer** class was employed to tokenize the training data. This step involved assigning a unique numerical index to each word based on its frequency in the corpus.

2. Sequence Generation:

- Sequences of numerical indices were generated for the training, validation, and test sets using the tokenizer.
- The sequences represented the tokenized version of the original text data.

3. Padding:

- To ensure uniformity in sequence lengths, padding was applied to sequences. A maximum sequence length of 200 words was imposed to standardize the input size for the models.
- The **pad_sequences** function was utilized for this purpose, adding zeros to the beginning of sequences if they were shorter than the specified length.

Processed Data Shapes

4. Data Shape Display:

- The processed data shapes were displayed to provide insights into the dimensions of the transformed datasets.
- The training set, after tokenization and padding, had a shape of (27300, 200), indicating 27,300 sequences with each sequence having a length of 200.
- Similarly, the validation set and test set had shapes of (9100, 200) each, adhering to the defined maximum sequence length.

Vanilla RNN (Recurrent Neural Network) Model

The Vanilla RNN model, a fundamental recurrent neural network architecture, was employed to perform text categorization on the preprocessed dataset. This section provides an overview of the model architecture, training process, and performance evaluation.

Model Architecture

The Vanilla RNN model was constructed using the following layers:

1. Embedding Layer:

- An embedding layer was utilized to convert the tokenized sequences into distributed representations. The layer had an input dimension of 10,000 words and an output dimension of 50.

2. SimpleRNN Layers:

- Two SimpleRNN layers were stacked, both comprising 50 units. The first layer returned sequences, while the second operated on the final output.

3. Dropout Layer:

- To mitigate overfitting, a dropout layer with a dropout rate of 0.5 was incorporated after the recurrent layers.

4. Dense Layers:

- Two dense layers followed, with the first having 50 units and a rectified linear unit (ReLU) activation function. The final dense layer, with softmax activation, produced predictions for the distinct categories.

Model Compilation

The model was compiled with the following specifications:

- **Optimizer:** Adam optimizer
- **Loss Function:** Sparse categorical crossentropy
- **Metrics:** Accuracy

Training Process

The training process involved the following steps:

1. Data Input:

- The tokenized and padded sequences (`X_train_padded`) served as input, with corresponding category labels (`y_train_np`).

2. Early Stopping:

- An early stopping callback was implemented, monitoring the validation loss. Training was halted if no improvement occurred after a patience of 5 epochs, with the best weights being restored.

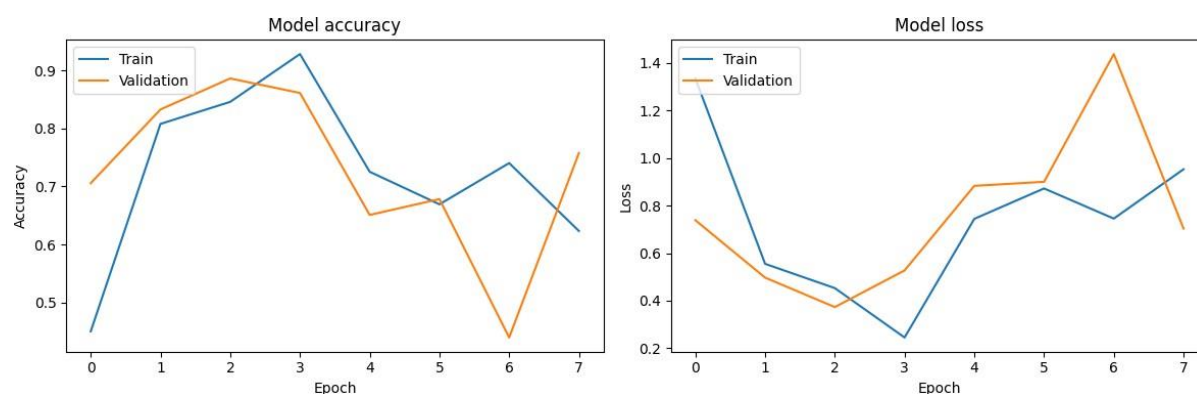
3. Epochs:

- The model was trained over 15 epochs, iteratively adjusting weights to optimize performance.

Performance Evaluation

The model's performance was evaluated on both the validation and test sets. The key metrics considered were training and validation accuracy, providing insights into the model's learning capabilities and generalization to unseen data.

Results



Accuracy: 88.55%

Classification Report:

	precision	recall	f1-score	support
0	0.85	0.72	0.78	1333
1	0.86	0.92	0.89	1256
2	0.98	0.96	0.97	1285
3	0.79	0.83	0.81	1323
4	0.87	0.95	0.91	1313
5	0.88	0.91	0.90	1262
6	0.97	0.91	0.94	1328
accuracy			0.89	9100
macro avg	0.89	0.89	0.89	9100
weighted avg	0.89	0.89	0.88	9100

Bidirectional LSTM (Long Short-Term Memory)

The Bidirectional LSTM model, an advanced recurrent neural network architecture, was implemented to further enhance the learning capabilities of the text categorization task. This section outlines the model architecture, training process, and performance evaluation.

Model Architecture

The Bidirectional LSTM model was constructed with the following architectural components:

1. Embedding Layer:

- An embedding layer was incorporated to transform tokenized sequences into distributed representations. The layer had an input dimension of 10,000 words and an output dimension of 128.

2. Bidirectional LSTM Layers:

- Two Bidirectional LSTM layers were stacked, each comprising 128 units. Bidirectional LSTM layers process input sequences in both forward and backward directions, capturing dependencies from both past and future contexts.

3. Reshaping Layer:

- A reshaping layer was introduced to transform the LSTM output into a three-dimensional shape, adding depth to the representations.

4. SpatialDropout1D Layer:

- A SpatialDropout1D layer enhanced dropout in the embedding layer, contributing to improved generalization.

5. Flatten Layer:

- The output was flattened before progressing to the dense layers.

6. Dense Layers:

- Two dense layers were added, with 256 units and ReLU activation in the first layer, followed by a dropout layer. The final dense layer, with softmax activation, produced category predictions.

Model Compilation

The model was compiled with the following specifications:

- **Optimizer:** Adam optimizer
- **Loss Function:** Sparse categorical crossentropy
- **Metrics:** Accuracy

Training Process

The training process involved the following steps:

1. Data Input:

- Tokenized and padded sequences (`X_train_padded`) served as input, with corresponding category labels (`y_train_np`).

2. Early Stopping:

- An early stopping callback was implemented, monitoring the validation loss. Training was halted if no improvement occurred after a patience of 9 epochs, with the best weights being restored.

3. Batch Size and Epochs:

- A batch size of 64 and 20 epochs were chosen for model training.

Performance Evaluation

The model's performance was evaluated on both the validation and test sets. Key metrics, including training and validation accuracy, were analyzed to assess the model's learning capabilities and generalization.

Results

Accuracy: 95.44%

Classification Report:

	precision	recall	f1-score	support
0	0.90	0.97	0.93	1333
1	0.96	0.95	0.96	1256
2	0.99	0.98	0.99	1285
3	0.95	0.89	0.92	1323
4	0.96	0.96	0.96	1313
5	0.95	0.97	0.96	1262
6	0.97	0.96	0.97	1328
accuracy			0.95	9100
macro avg	0.96	0.95	0.95	9100
weighted avg	0.96	0.95	0.95	9100

Multinomial Naive Bayes Model

The Multinomial Naive Bayes model, a probabilistic classifier, played a pivotal role in categorizing Arabic text into predefined topics. This section outlines the methodology, model training process, and evaluation metrics.

Methodology

The Multinomial Naive Bayes model operates on the principle of Bayes' theorem, assuming independence between features. It is particularly well-suited for text classification tasks, making it a suitable baseline model for this project.

Model Training

1. Count Vectorization:

- The CountVectorizer was utilized to convert the text data into numerical vectors. This process involved representing each document as a vector of word frequencies.

2. Vectorized Training Data:

- The training data was transformed into numerical vectors using the fitted CountVectorizer. The resulting matrix represented the frequency of each word in the training documents.

3. Multinomial Naive Bayes Model:

- A Multinomial Naive Bayes model was instantiated and fitted on the vectorized training data. This model is well-suited for discrete data, making it suitable for the count-based features obtained through vectorization.

Prediction and Evaluation

4. Vectorization of Validation and Test Sets:

- The same CountVectorizer used for training was applied to transform the validation and test sets into numerical vectors.

5. Validation Set Evaluation:

- The model predicted categories for the validation set, and metrics such as accuracy and the classification report were calculated to assess performance.

6. Test Set Evaluation:

- The model's predictive capabilities were evaluated on the test set, and accuracy along with a detailed classification report was generated.

Results

Validation Set Metrics:

Accuracy: 0.9563

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.96	0.95	1346
1	0.90	0.98	0.94	1272
2	1.00	0.98	0.99	1332
3	0.96	0.92	0.94	1317
4	0.98	0.97	0.97	1305
5	0.95	0.98	0.97	1289
6	0.99	0.90	0.94	1239
accuracy			0.96	9100
macro avg	0.96	0.96	0.96	9100
weighted avg	0.96	0.96	0.96	9100

Comparative Analysis

Model	Accuracy	Precision	Recall	F1-Score
Vanilla RNN	88.55%	0.89	0.89	0.88
Bidirectional LSTM	95.44%	0.96	0.95	0.95
Multinomial Naive Bayes	95.63%	0.96	0.96	0.96

Observations:

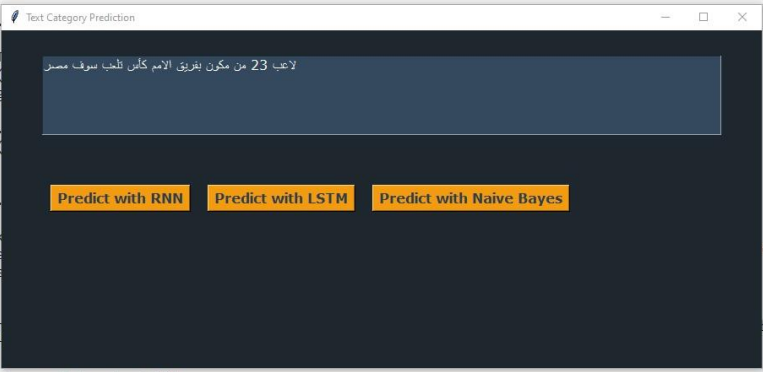
- 1. Accuracy:**
 - Both the Bidirectional LSTM and Multinomial Naive Bayes models outperform the Vanilla RNN in terms of accuracy, achieving 95.44% and 95.63%, respectively, compared to 88.55% for the Vanilla RNN.
- 2. Precision, Recall, F1-Score:**
 - The Bidirectional LSTM and Multinomial Naive Bayes models consistently demonstrate higher precision, recall, and F1-score across all categories compared to the Vanilla RNN.
- 3. Strengths:**
 - The Bidirectional LSTM and Multinomial Naive Bayes models exhibit strengths in capturing intricate patterns and dependencies within the data, leading to superior performance in classification metrics.
- 4. Simplicity vs. Complexity:**
 - While the Multinomial Naive Bayes model achieves comparable results to the Bidirectional LSTM, it does so with a simpler architecture. The Bidirectional LSTM, with its more complex structure, demonstrates superior accuracy.
- 5. Trade-offs:**
 - The Vanilla RNN, while being the simplest, lags behind in accuracy compared to the other two models. The trade-off between model complexity and performance is evident.

Arabic Text Mastermind: Intuitive GUI Edition

```
jupyter nlp-project (2) Last Checkpoint: 3 hours ago (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

predicted_category_index = np.argmax(predictions[0])
predicted_category = categories[predicted_category_index]

result, 

root = tk.Tk()
root.title("Text Category Prediction")
root.config(bg='black')

frame = tk.Frame(root, bg='black')
frame.pack()

text_entry = tk.Entry(frame, bg='black', fg='white')
text_entry.pack()

result = tk.Label(frame, bg='black', fg='white')
result_label = tk.Label(frame, bg='black', fg='white')

# Function to predict using RNN
predict_rnn = lambda: predict_category("RNN")
predict_rnn_button = tk.Button(frame, text="Predict with RNN", command=predict_rnn, bg='black', fg='white')
predict_rnn_button.pack(side=tk.LEFT, padx=10)

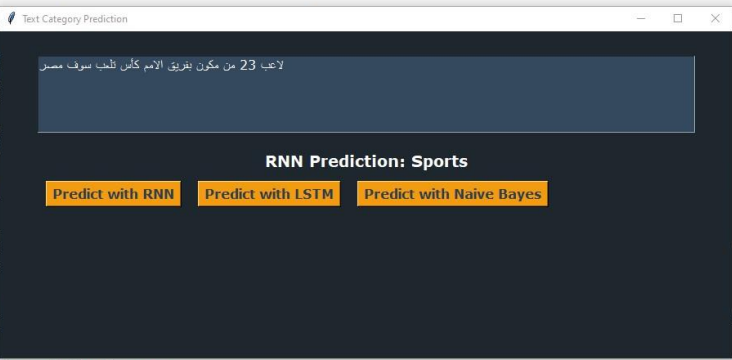
# Function to predict using LSTM
predict_lstm_button = tk.Button(frame, text="Predict with LSTM", command=lambda: predict_category("LSTM"), bg='black', fg='white')
predict_lstm_button.pack(side=tk.LEFT, padx=10)

# Function to predict using Naive Bayes
predict_nb_button = tk.Button(frame, text="Predict with Naive Bayes", command=lambda: predict_category("NaiveBayes"), bg='black', fg='white')
predict_nb_button.pack(side=tk.LEFT, padx=10)
```

```
jupyter nlp-project (2) Last Checkpoint: 3 hours ago (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

predicted_category_index = np.argmax(predictions[0])
predicted_category = categories[predicted_category_index]

result, 

root = tk.Tk()
root.title("Text Category Prediction")
root.config(bg='black')

frame = tk.Frame(root, bg='black')
frame.pack()

text_entry = tk.Entry(frame, bg='black', fg='white')
text_entry.pack()

result = tk.Label(frame, bg='black', fg='white')
result_label = tk.Label(frame, bg='black', fg='white')

# Function to predict using RNN
predict_rnn = lambda: predict_category("RNN")
predict_rnn_button = tk.Button(frame, text="Predict with RNN", command=predict_rnn, bg='black', fg='white')
predict_rnn_button.pack(side=tk.LEFT, padx=10)

# Function to predict using LSTM
predict_lstm_button = tk.Button(frame, text="Predict with LSTM", command=lambda: predict_category("LSTM"), bg='black', fg='white')
predict_lstm_button.pack(side=tk.LEFT, padx=10)

# Function to predict using Naive Bayes
predict_nb_button = tk.Button(frame, text="Predict with Naive Bayes", command=lambda: predict_category("NaiveBayes"), bg='black', fg='white')
predict_nb_button.pack(side=tk.LEFT, padx=10)
```